

Week 1 | Python Programming Lab Tasks

Task 1: Numerical Transformation Using List Data Structure

Objective

To apply conditional logic and functional transformation on numerical data using Python list data structure.

Instructions

Create a Python program that:

1. Accepts a list of numerical values (hardcoded or user-defined).
2. Apply conditional transformations such that:
 - All even numbers are squared
 - All odd numbers are cubed
3. Stores results in separate lists
4. Merges both lists into a **single sorted list**

Task 2: Dictionary-Based Information Mapping and Ranking

Objective

To model structured data using dictionaries and perform value-based analysis.

Instructions

Create a dictionary representing at least five entities (e.g., students, products, sensors), where each entity stores:

- Name
- Category
- Numerical score/value

Your program must:

1. Insert entries dynamically
2. Identify entities with:
 - Maximum value
 - Minimum value
3. Display all entities sorted by value

Task 3: Functional Programming and Lambda-Based Data Normalization

Objective

To implement functional programming paradigms and apply normalization for numerical stability.

Instructions

Write a Python program that:

1. Defines lambda functions for:
 - Absolute difference between two values
 - Min–Max normalization of numerical values
2. Applies these functions on a numerical list using:
 - `map()`
 - `filter()`

Constraint

Normalization must rescale values strictly between 0 and 1 using Min–Max normalization.

NOTE:

Normalization means **rescaling data to a common range**, usually **0 to 1**, so that:

- Large values do not dominate smaller values
- Different features become comparable
- Algorithms behave more stably and fairly

Simple idea:

Convert values like [10, 50, 100] into something like [0.0, 0.44, 1.0]

Why do we normalize data?

Imagine two features:

Feature	Range
Age	0 – 100
Income	10,000 – 1,000,000

If we don't normalize:

- Income will dominate calculations
- Distance-based methods behave incorrectly
- Learning becomes unstable

Normalization ensures all features contribute equally.

Most Common Normalization Method (Min–Max Normalization)

Formula:

What this does:

- Smallest value $\rightarrow 0$
- Largest value $\rightarrow 1$
- Everything else $\rightarrow$ between 0 and 1

Task 4: Multi-Dimensional Data Representation Using NumPy

Objective

To explore multi-dimensional numerical data structures and extract metadata.

Instructions

Using NumPy:

1. Generate a 6×4 random matrix
2. Display:
 - Shape
 - Size
 - Data type
3. Compute:
 - Column-wise mean
 - Row-wise maximum
4. Identify the index of the maximum element in the matrix

Task 5: Data Ranking and Comparative Analysis Using NumPy

Objective

To rank entities based on computed performance metrics.

Instructions

Create a NumPy array representing performance scores of multiple systems, where:

- Rows represent systems
- Columns represent test cases

Your program must:

1. Compute the average score per system
2. Rank systems using `argsort()`
3. Display systems from best to worst

Functions:

- Statistical operations
- `mean()`
- `argsort()`
- Array indexing

Task 6: Pattern Discovery Through Statistical Analysis

Objective

To analyze variation and aggregate behavior in multi-dimensional data.

Instructions

Using NumPy:

1. Generate a 100×3 random dataset
2. Normalize each column between 0 and 1
3. Identify:
 - The column with highest variance
 - The row with maximum total value

*This task introduces **data variability analysis**, a core AI preprocessing concept.*

Task 7: List Manipulation and Type-Based Filtering

Objective

To apply slicing, cloning, and type-based filtering on mixed data.

Instructions

Create a list containing areas of different house sections with the following values:

112.0, 113.5, 189.5, 789.0, 230.2, "20.0"

Perform the following:

1. Clone the list into `float_area`
2. Retain only float values in `float_area`
3. Extract string values into `str_area`
4. Print all lists using slicing
5. Display the bedroom area along with its label using slicing

Task 8: Recursive Deep Flattening of Nested Lists

Objective

To solve a recursive data-structure problem using Python.

Instructions

Given a nested list representing house extensions:

```
['bed',112.0,'hall',113.5,'bath',189.5,'kit',789.0,'liv',230.0,  
['pool',112.7,117.9],['grg',112.0]]
```

Write a Python program to deep-flatten the list such that all nested elements are extracted into a single-level list.

Task 9: Dictionary-to-List Transformation

Objective

To extract and restructure data from dictionaries into multiple list formats.

Instructions

Given a list of dictionaries representing students, create:

```
students = [  
    {'student_id': 1, 'name': 'Jean Castro', 'class': 'V'},  
    {'student_id': 2, 'name': 'Lula Powell', 'class': 'V'},
```

```
{'student_id': 3, 'name': 'Brian Howell', 'class': 'VI'},  
]
```

1. A list containing all values
2. A list containing [student_id, name]
3. A list containing [name, class]
4. Perform inverse transformations where applicable

*This task emphasizes **data restructuring**, a key step in AI pipelines.*

Task 10: Set-Based Subset Analysis

Objective

To identify dominant data groups using set theory and subset relations.

Instructions

Given a list of favorite food items per person:

```
Fav_food_items = [  
    ['pizza','burger','hotdogs'],  
    ['pasta','hotdogs'],  
    ['pizza'],  
    ['burger','hotdogs'],  
    ['rice','pasta'],  
    ['pasta']  
]
```

Write a Python function that:

1. Determines which lists are not subsets of any other list
2. Returns their indices in ascending order

Task 11: Frequency Analysis Using Dictionaries

Objective

To compute frequency distributions for unstructured categorical data.

Instructions

Given a list of user activity logs:

```
activities = ['login','logout','login','purchase','login','purchase','logout','login']
```

Write a program that:

1. Counts frequency of each activity
2. Sorts activities by frequency (descending)
3. Identifies the most and least frequent activities

Task 12: Constraint-Based Problem Solving

Objective

To solve a constraint-driven data processing problem using core Python constructs.

Problem Statement

You are given a dataset representing system resource usage:

```
usage = [  
    {'cpu':45, 'memory':70},  
    {'cpu':85, 'memory':90},  
    {'cpu':60, 'memory':40},  
    {'cpu':95, 'memory':88},  
    {'cpu':30, 'memory':20}  
]
```

Write a program that:

1. Identifies systems where:
 - CPU usage > 80 **AND**
 - Memory usage > 75
2. Normalizes CPU and memory values between 0-1
3. Ranks systems based on combined normalized scores

Task 13: Extract Values from Dictionaries and Create List

Write a Python program to extract values from a given dictionaries and create a list of lists from those values.

Original Dictionary:

```
[{'student_id': 1, 'name': 'Jean Castro', 'class': 'V'}, {'student_id': 2, 'name': 'Lula Powell', 'class': 'V'}, {'student_id': 3, 'name': 'Brian Howell', 'class': 'VI'}, {'student_id': 4, 'name': 'Lynne Foster', 'class': 'VI'}, {'student_id': 5, 'name': 'Zachary Simon', 'class': 'VII'}]
```

Extract values from the said dictionary and create a list of lists using those values:

```
[[1, 'Jean Castro', 'V'], [2, 'Lula Powell', 'V'], [3, 'Brian Howell', 'VI'], [4, 'Lynne Foster', 'VI'], [5, 'Zachary Simon', 'VII']]  
[[1, 'Jean Castro'], [2, 'Lula Powell'], [3, 'Brian Howell'], [4, 'Lynne Foster'], [5, 'Zachary Simon']]  
[['Jean Castro', 'V'], ['Lula Powell', 'V'], ['Brian Howell', 'VI'], ['Lynne Foster', 'VI'], ['Zachary Simon', 'VII']]  
And vice versa
```

Task 14: Tuple

Build a ten-element tuple of random numbers and then sort the tuple in increasing order without using built-in function. Remember the result should be a tuple.

Task 15: Swapping

Write a NumPy program to create a three-dimension array with shape (3,5,4) and set to a variable, then swap rows and columns of a given array in reverse order.